

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎

2.8 内存大数据分析系统PowerDrill

Dremel的推出，在一定程度上减轻了Google内部对于MapReduce系统的依赖。但随着数据量的增加和对数据分析时效性的要求越来越高，Dremel在有些场景下也显得不太适用，为此Google研发了内部代号为PowerDrill的系统。

该系统从2008年年底开始部署，2009年中旬开始在Google内部公开使用。到了2012年，Google才在VLDB会议上公开了PowerDrill的部分实现。

根据Google自己的说法，该系统在针对相对特定的数据集上，可以比Dremel处理得更快。由于Google主要公开的是PowerDrill的存储部分，因此本节主要介绍PowerDrill为了达到更快的处理速度所采用的存储方案及相关的优化等。

2.8 内存大数据分析系统PowerDrill

- ▶ 2.8.1 产生背景与设计目标
- 2.8.2 基本数据结构
- 2.8.3 性能优化
- 2.8.4 性能分析与对比

2.8 内存大数据分析系统PowerDrill

1

用户对于实时的交互式数据查询和分析一直都有很高的要求，尤其是在一些数据探索（data exploration）的场景

2

数据探索：在完成一项任务之前，需要先向系统发出请求，根据得到结果来修正查询内容，并再次向系统发出新的查询，如此反复的过程可能要进行很多次。

3

MapReduce无法实现这种程度的交互式查询。Dremel可以在一定程度上实现实时的交互式查询，但是随着数据规模的增大和Google内部对ad hoc查询（即席查询）需求的增多，Google设计和开发了新的交互式查询系统PowerDrill。

2.8 内存大数据分析系统PowerDrill

1

ad hoc查询（即席查询）是用户根据自己的需求，灵活地选择查询条件，系统根据用户的选择生成相应的统计结果。

2

通常的查询在系统设计和实施时是已知的，所以我们可以系统在构建时，通过建立索引、分区等技术来优化这些查询，达到提升查询效率的目的。

3

但ad hoc查询是用户在使用时产生的，系统无法预先优化这些查询，因此，需要通过其他技术手段来实现高效的ad hoc查询。

例如，在一个销售数据分析系统中，通常会有一些预定义的报表，如每月销售报表、各地区销售排名等。但如果销售经理突然想要知道上个季度中，销售额在 10 万 - 20 万之间的产品类别以及对应的销售人员，而且没有现成的报表满足这个需求，这时候发起的查询就是即席查询。

2.8 内存大数据分析系统PowerDrill

● 产生背景与设计目标

通过对内部数据和查询类型的分析，PowerDrill开发团队得出了两个假设结论，整个系统的开发和优化都基于这两个假设。

两个假设结论

- (1) 绝大多数的查询是类似和一致的；
- (2) 存储系统中的表只有一小部分是经常被使用的，绝大部分的表使用频率不高。

2.8 内存大数据分析系统PowerDrill

● 产生背景与设计目标

在两个假设的指导下，为了实现高效的查询，PowerDrill开发团队主要考虑两方面的内容：

考虑两方面的内容

- (1) 如何尽可能在查询中略去不需要的数据分块；
- (2) 如何尽可能地减少数据在内存中的占用，占用越少意味着越多的数据可以被加载进内存中处理。

2.8 内存大数据分析系统PowerDrill

● 产生背景与设计目标

与Dremel数据处理方式不同的是，PowerDrill需要尽可能地将数据加载至内存，在某种程度上，其计算方式更接近**内存计算**。

PowerDrill整个系统实际分为三个部分

Web UI

一个抽象层

列式存储

1

Web UI：用于和用户进行交互；

2

一个抽象层：用户的命令会被转换成SQL，然后根据不同的需求，这些命令会被发送至不同的终端，比如Dremel，Bigtable等；

3

列式存储

2.8 内存大数据分析系统PowerDrill

2.8.1 产生背景与设计目标

▶ 2.8.2 基本数据结构

2.8.3 性能优化

2.8.4 性能分析与对比

2.8 内存大数据分析系统PowerDrill

列式存储主要有两个好处：减少查询涉及的数据量以及便于数据压缩。传统的关系数据库一般通过**索引**来减少查询中用到的数据量，但是在ad hoc查询这样的场景下，索引基本不起作用。

PowerDrill采用的方式是对数据进行**分块**，对块数据设计巧妙的数据结构，使得在查询时可以确定哪些块不需要，可以直接被略去，这样就大大减少了所需的数据量。

2.8 内存大数据分析系统PowerDrill

● 基本数据结构

下图是一张存储搜索关键字 (search string) 的表，阐述了PowerDrill采用的数据结构，简单来说就是一个双层数据字典结构。

global-dictionary dict		chunk 0			chunk 1			chunk 2		
id	search string	chunk-dict <i>ch₀.dict</i>		elements <i>ch₀.elems</i>	chunk-dict <i>ch₁.dict</i>		elements <i>ch₁.elems</i>	chunk-dict <i>ch₂.dict</i>		elements <i>ch₂.elems</i>
		id	global-id		id	global-id		id	global-id	
0	ab in den Urlaub	0	1	3	0	0	5	0	1	0
1	amazon	1	2	2	1	1	2	1	3	0
2	cheap tickets	2	4	0	2	5	1	2	5	2
3	chaussures	3	5	4	3	6	4	3	10	4
4	cheap flights	4	12	0	4	7	3	4	10	3
5	ebay			0	5	8	0			4
6	faschingskosttime			0			0			4
7	immobilienscout			2			1			5
8	karnevalskosttime			1			5			2
9	la redoute			3			5			1
10	pages jaunes			2						
11	voyages snfc									
12	yellow pages									

2.8 内存大数据分析系统PowerDrill

● 基本数据结构

图中假设表的数据已经被分成3块，最左侧是一个全局字典表，存储的是全局id (global-id) 和搜索关键字 (search string) 的对应关系。

global-dictionary dict		chunk 0			chunk 1			chunk 2		
id	search string	chunk-dict <i>ch₀.dict</i>		elements <i>ch₀.elems</i>	chunk-dict <i>ch₁.dict</i>		elements <i>ch₁.elems</i>	chunk-dict <i>ch₂.dict</i>		elements <i>ch₂.elems</i>
0	ab in den Urlaub	id	global-id	3	id	global-id	5	id	global-id	0
1	amazon									
2	cheap tickets	0	1	2	0	0	2	0	1	0
3	chaussures	1	2	0	1	1	1	1	3	0
4	cheap flights	2	4	4	2	5	4	2	5	2
5	ebay	3	5	0	3	6	3	3	10	4
6	faschingskosttime	4	12	0	4	7	0	4	10	3
7	immobilienscout			0	5	8	0			4
8	karnevalskosttime			2			1			4
9	la redoute			1			5			5
10	pages jaunes			3			5			2
11	voyages snfc			2			5			1
12	yellow pages									

全局字典表

存储全局id和搜索关键字的对应关系

3个块的数据

2.8 内存大数据分析系统PowerDrill

● 基本数据结构

每个数据块主要由两部分组成：**块字典**和**块元素**。

在具体查询时，需要完成**两层的数据映射**才能得到真实的数据值。

global-dictionary dict		chunk 0			chunk 1			chunk 2		
id	search string	chunk-dict <i>ch₀.dict</i>		elements <i>ch₀.elems</i>	chunk-dict <i>ch₁.dict</i>		elements <i>ch₁.elems</i>	chunk-dict <i>ch₂.dict</i>		elements <i>ch₂.elems</i>
0	ab in den Urlaub	id	global-id		id	global-id		id	global-id	
1	amazon	0	1	3	0	0	5	0	1	0
2	cheap tickets	1	2	2	1	1	2	1	3	0
3	chaussures	2	4	0	2	5	1	2	5	2
4	cheap flights	3	5	4	3	6	4	3	10	4
5	ebay	4	12	0	4	7	3	4	10	3
6	faschingskosttime			0	5	8	0			4
7	immobilienscout			0			0			4
8	karnevalskosttime			2			1			5
9	la redoute			1			5			2
10	pages jaunes			3			5			1
11	voyages snfc			2						
12	yellow pages									

全局字典表

存储全局id和搜索关键字的对应关系

3个块的数据

块字典记录的是块id (chunk-id) 和全局id的映射关系

2.8 内存大数据分析系统PowerDrill

● 基本数据结构

每个数据块主要由两部分组成：块字典和块元素。

在具体查询时，需要完成两层的数据映射才能得到真实的数据值。

global-dictionary dict		chunk 0			chunk 1			chunk 2		
id	search string	chunk-dict <i>ch₀.dict</i>		elements <i>ch₀.elems</i>	chunk-dict <i>ch₁.dict</i>		elements <i>ch₁.elems</i>	chunk-dict <i>ch₂.dict</i>		elements <i>ch₂.elems</i>
0	ab in den Urlaub	id	global-id	3	id	global-id	5	id	global-id	0
1	amazon			2			2			0
2	cheap tickets			0			1			2
3	chaussures			4			4			4
4	cheap flights			0			3			3
5	ebay			0			0			4
6	faschingskosttime	3	5	2			0	10	10	5
7	immobilienscout	4	12	1	4	7	5	4	10	2
8	karnevalskosttime			3	5	8	5			1
9	la redoute			2						
10	pages jaunes			2						
11	voyages snfc			1						
12	yellow pages			2						

全局字典表

存储全局id和搜索关键字的对应关系

3个块的数据

块字典 记录的是块id (chunk-id) 和全局id的映射关系

块元素 记录的是块中存储数据的块id (注意不是全局id)

2.8 内存大数据分析系统PowerDrill

● 基本数据结构

例如，chunk 0 中第一个块id为3，查块字典得到其全局id为5，再查询全局字典得到其真实值为ebay。反过来，从真实值查询其对应的块id也是类似的过程。

global-dictionary dict		chunk 0			chunk 1			chunk 2		
id	search string	chunk-dict <i>ch₀.dict</i>		elements <i>ch₀.elems</i>	chunk-dict <i>ch₁.dict</i>		elements <i>ch₁.elems</i>	chunk-dict <i>ch₂.dict</i>		elements <i>ch₂.elems</i>
0	ab in den Urlaub	id	global-id	3	id	global-id	5	id	global-id	0
1	amazon									
2	cheap tickets	0	1	2	0	0	2	0	1	0
3	chaussures	1	2	0	1	1	1	1	3	0
4	cheap flights	2	4	0	2	5	4	2	5	2
5	ebay	3	5	4	3	6	3	3	10	4
6	fashingskosttime	4	12	0	4	7	0	4	10	3
7	immobilienscout			0	5	8	0			4
8	karnevalskosttime			2			1			4
9	la redoute			1			5			5
10	pages jaunes			3			5			2
11	voyages snfc			2			5			1
12	yellow pages									

全局字典表

存储全局id和搜索关键字的对应关系

3个块的数据

块字典 记录的是块id (chunk-id) 和全局id的映射关系

块元素 记录的是块中存储数据的块id (注意不是全局id)

2.8 内存大数据分析系统PowerDrill

● 基本数据结构

例如，chunk 0 中第一个块id为3，查全局字典得到其真实值为ebay。反过来，从真实值

global-dictionary dict		chunk 0		
id	search string	chunk-dict $ch_0.dict$		elements $ch_0.elems$
0	ab in den Urlaub	id	global-id	
1	amazon			
2	cheap tickets	0	1	3
3	chaussures	1	2	2
4	cheap flights	2	4	0
5	ebay	3	5	4
6	fashingskosttime	4	12	0
7	immobilienscout			0
8	karnevalskosttime			2
9	la redoute			1
10	pages jaunes			3
11	voyages snfc			2
12	yellow pages			1

块元素	global-id	数据
3	5	ebay
2	4	cheap flights
0	1	ab in den Urlaub
4	12	yellow pages
0	1	amazon
0	1	amazon
2	4	cheap flights
1	2	cheap tickets
3	5	ebay
2	4	cheap flights

全局字典

呈。

elements

$ch_2.elems$

0

0

2

4

3

4

4

5

2

1

2.8 内存大数据分析系统PowerDrill

2.8.1 产生背景与设计目标

2.8.2 基本数据结构

► 2.8.3 性能优化

2.8.4 性能分析与对比

2.8 内存大数据分析系统PowerDrill

• 1. 数据分块

背景

传统的索引对于PowerDrill的查询场景作用不是很大，因此一个很自然的考虑就是对数据进行**分块**，**过滤**查询中不需要的数据块来减少数据量

方法

常见的分区方法有**范围分区**、**散列分区**等。PowerDrill实际采用的是一种**组合范围分区**方法。

步骤

领域专家确定若干个划分的域



利用这几个**域**对数据进行划分



每个块的**行数**达到阈值时就停止划分

局限

PowerDrill采用的数据分块方法简单实用，但是由于域的确定**需要领域专家**，因此这种方法在实际使用中还有一定的局限性

2.8 内存大数据分析系统PowerDrill

● 2. 存储方案选择：数据编码的优化

- 对于不同的块，如果我们可以确定块中**不同值的数量**，那么就可以根据这个数量值来选择可变的**比特位**来**记录块id**
- 统计一组数中不同值的个数有一个专有名词，称为 **“基数估计”**
- 对于**小规模**的数据集，可以比较**容易**地统计出精确的基数。但是在**大数据的**环境下，精确的基数统计非常**耗时**，因此能保证一定精度的**基数估计**就可以满足实际的需求。
- 基数估计的方法很多，大多利用了**散列函数**的一些特性，Google内部使用的是一种称为**Hyperloglog**的基数估计方法的变种。

2.8 内存大数据分析系统PowerDrill

● 3. 全局字典非常庞大——优化

优化中主要利用
两个特性

全局字典是有序的

排序后的数据常常有共同的前缀

实际使用中为了进一步减少查询中需要加载到内存的全局字典，对全局字典又进行了分块

对每个全局字典块还会维护一个布隆过滤器 (bloom filter) 来快速确定某个值是否在字典中。

2.8 内存大数据分析系统PowerDrill

● 4. 压缩算法

Google曾经对一些主流的压缩算法做过简单的测试，如下图：

算法	压缩后剩余数据百分比	编码速度	解码速度
GZIP	13.4%	21 MB/s	118 MB/s
LZO	20.5%	135 MB/s	410 MB/s
Zippy/Snappy	22.2%	172 MB/s	409 MB/s

- 不管压缩算法的解压速度多快，总会消耗一定的物理资源与时间。对此PowerDrill采用了一种**冷热数据**分别对待的策略。
- **冷数据**是指那些访问频率很低的数据，**热数据**是指那些访问频率非常高的数据。

2.8 内存大数据分析系统PowerDrill

● 4. 压缩算法

Google曾经对一些主流的压缩算法做过简单的测试，如下图：

算法	压缩后剩余数据百分比	编码速度	解码速度
GZIP	13.4%	21 MB/s	118 MB/s
LZO	20.5%	135 MB/s	410 MB/s
Zippy/Snappy	22.2%	172 MB/s	409 MB/s

- 在冷热数据切换策略中，比较常用的是**LRU**算法。PowerDrill开发团队采用了**启发式的缓存策略**来代替原始的LRU算法。

2.8 内存大数据分析系统PowerDrill

● 4. 压缩算法

Google曾经对一些主流的压缩算法做过简单的测试，如下图：

算法	压缩后剩余数据百分比	编码速度	解码速度
GZIP	13.4%	21 MB/s	118 MB/s
LZO	20.5%	135 MB/s	410 MB/s
Zippy/Snappy	22.2%	172 MB/s	409 MB/s

- **LRU** (Least Recently Used, 最近最少使用) 策略会记录数据被访问的时间。当数据被访问时，其访问时间就会被更新。当缓存（或存储区域）已满，需要引入新的数据时，LRU 会选择**最近最少**被访问的数据进行替换。也就是说，在缓存中的所有数据里，距离上次被访问时间**最久**的数据会被判定为“**冷数据**”，并被新数据替换。

2.8 内存大数据分析系统PowerDrill

● 4. 压缩算法

Google曾经对一些主流的压缩算法做过简单的测试，如下图：

算法	压缩后剩余数据百分比	编码速度	解码速度
GZIP	13.4%	21 MB/s	118 MB/s
LZO	20.5%	135 MB/s	410 MB/s
Zippy/Snappy	22.2%	172 MB/s	409 MB/s

- 启发式缓存策略基于一定经验规则和局部最优原则来决定数据存储和替换的方法。它不像传统的基于固定规则（如 LRU）的缓存策略那样刻板，而是综合考虑多种因素，以更灵活地适应不同的应用场景。
例如，基于频率和时间的策略，基于数据大小和价值的策略，预测性缓存策略。

2.8 内存大数据分析系统PowerDrill

● 5. 行的重排

在列存储中，对行进行适当的重排不会影响结果，且会提升压缩效率。



- 从直观上感觉，肯定是右侧的数据顺序更利于数据压缩。为什么？

2.8 内存大数据分析系统PowerDrill

● 5. 行的重排

数据压缩的算法有很多，比较常用的一种称为**游程编码**（Run-Length Encoding, **RLE**），又称行程长度编码，其好处是压缩和解压缩都非常快。

RLE算法将数据压缩成**三元组**的形式，比如

AAAABBBBBBCCCCCC

这样一个包含**16**条数据的某列数值。

RLE算法会将它压缩成形如“[实际值，起始位置，个数]”的三元组，即

[A, 1, 4]、[B, 5, 6]、[C, 11, 5]

从而比原始数据占用更少的空间。

由于RLE算法的特点，相邻数据之间越**相似**，压缩效果越好，因此，数据往往需要重新排序，从而得到更好的结果。

2.8 内存大数据分析系统PowerDrill

• 5. 行的重排

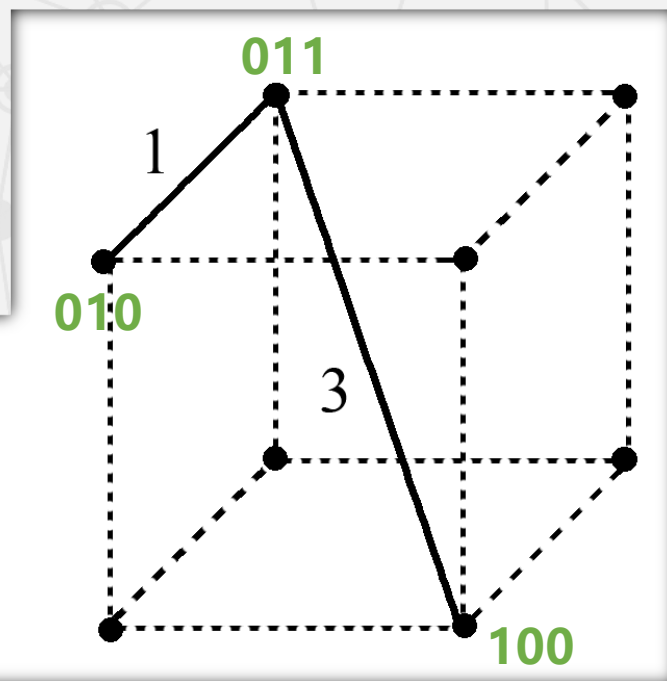
如何重排？ 数据重排的过程等效于著名的TSP（旅行商）问题，也就是说，这是一个NP困难问题。

首先，我们来看TSP（旅行商）问题：

TSP (Traveling Salesman Problem) 是**组合优化**领域中的一个经典问题。

简单来说，给定一组城市以及城市之间的**距离**，要求找到一条经过**所有**城市且每个城市只经过**一次**的**最短**路径。

0	1	0
0	1	1
1	0	0



2.8 内存大数据分析系统PowerDrill

• 5. 行的重排

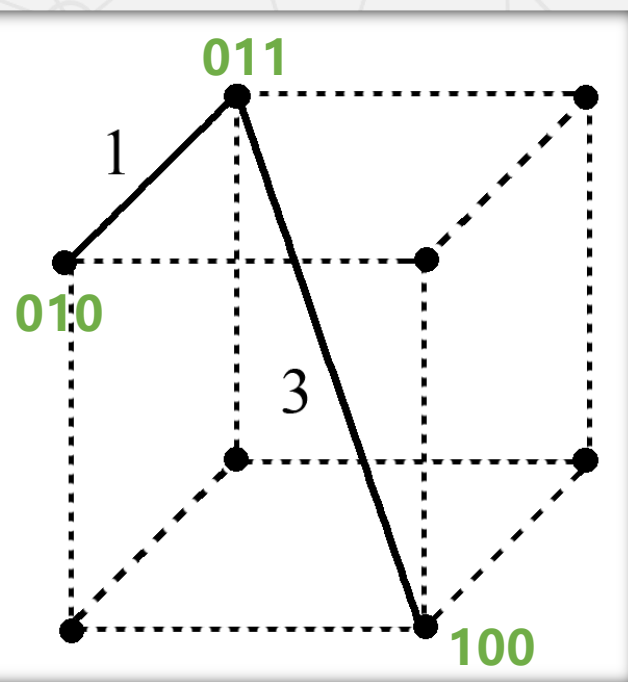
数据重排的过程等效于著名的TSP（旅行商）问题，也就是说，这是一个NP困难问题。

那NP困难问题又是什么呢？

在计算复杂性理论中，**NP 问题**的计算复杂度是多项式时间，如 $O(n^2)$ 、 $O(n^3)$ 等， n 为问题规模。

而**NP困难**问题的计算复杂度是指数时间，如 $O(2^n)$ 、 $O(3^n)$ 等。

0	1	0
0	1	1
1	0	0



2.8 内存大数据分析系统PowerDrill

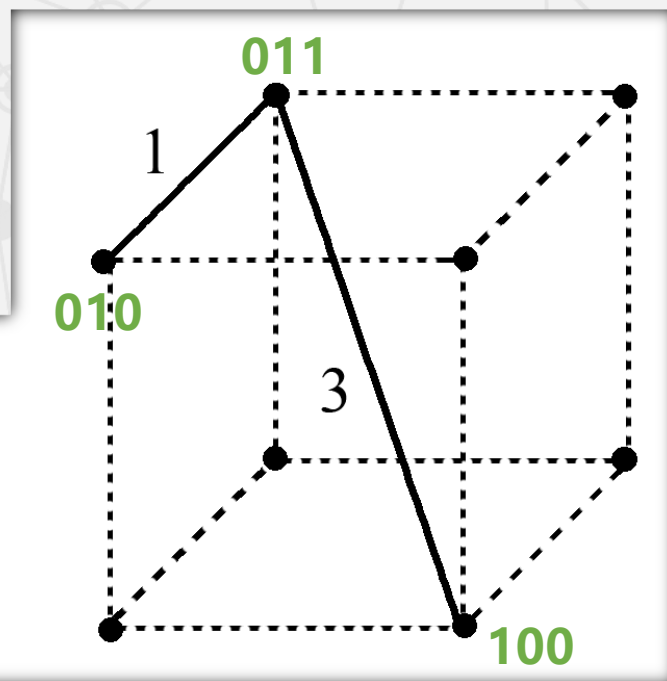
● 5. 行的重排

数据重排的过程等效于著名的TSP（旅行商）问题，也就是说，这是一个NP困难问题。

TSP 是 NP - Hard 问题，随着城市数量的增加，找到最优解的计算复杂度呈指数级增长。

虽然可以用一些启发式算法在合理的时间内找到较优解，但要确定绝对的最优解是非常困难的。

0	1	0
0	1	1
1	0	0



2.8 内存大数据分析系统PowerDrill

• 5. 行的重排

数据重排的过程等效于著名的TSP（旅行商）问题，也就是说，这是一个NP困难问题。为什么数据重排的过程等效于TSP问题呢？

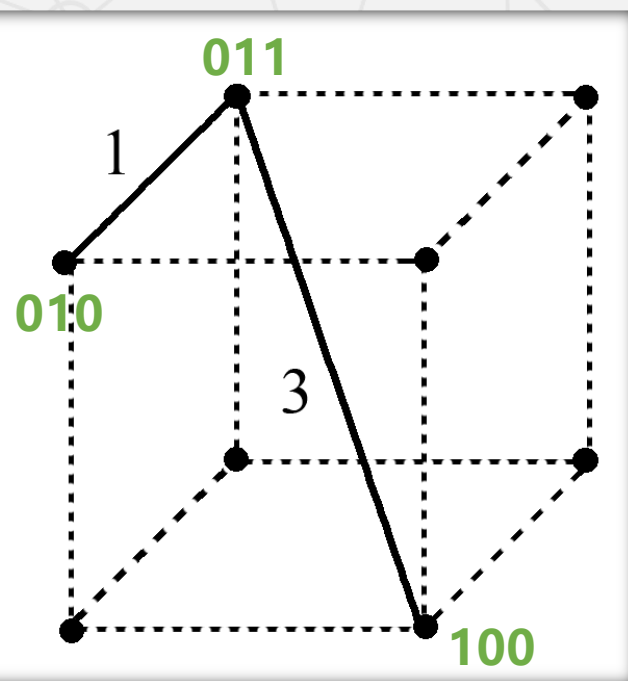
首先，我们来看**汉明距离**（Hamming distance）：

在信息论中，两个等长字符串之间的**汉明距离**是两个字符串对应位置的**不同字符的个数**。

010与011的汉明距离为1.

将每一行字符串看做汉明空间的一个点，则右图表示这三个点的访问顺序。

0	1	0
0	1	1
1	0	0



2.8 内存大数据分析系统PowerDrill

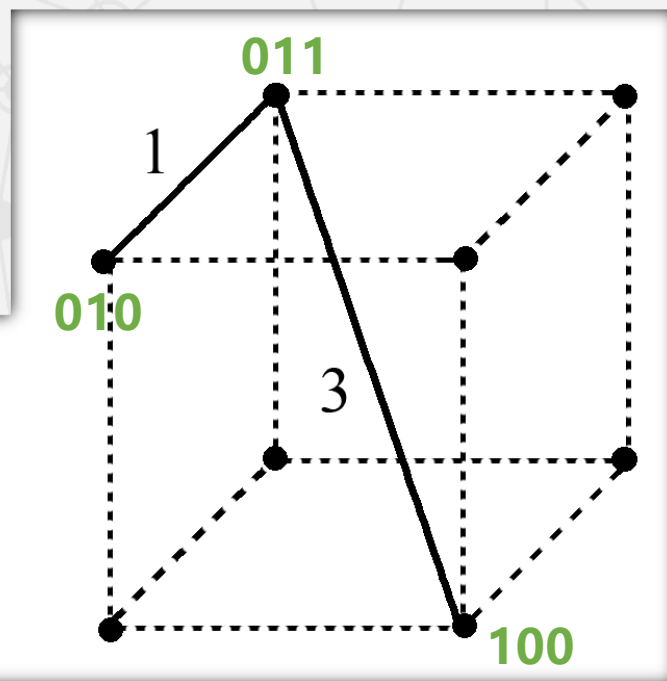
● 5. 行的重排

从010到011的距离为1，再到100的距离为3，总距离为4. 这也是使得这3个点在汉明空间取得最小总距离的访问顺序。

根据RLE游程编码算法，保证相邻两行之间的距离之和最小，就能保证总距离最小，继而保证高效的压缩。

这相当于寻找一种重排方案，使得重排后的数据从第一行一直遍历到最后一行，每次计算相邻两行的距离，最后累加值最小。

0	1	0
0	1	1
1	0	0



PowerDrill对数据分块时，选定的那几个域按照字典序进行排序，来得到重排结果。

2.8 内存大数据分析系统PowerDrill

2.8.1 产生背景与设计目标

2.8.2 基本数据结构

2.8.3 性能优化

► 2.8.4 性能分析与对比

2.8 内存大数据分析系统PowerDrill

- 我们比较关注的两组数据



在查询过程中，平均**92.41%**的数据被略去

5.02%的数据会直接被缓存命中

一般仅须扫描**2.66%**的数据即可得到查询结果

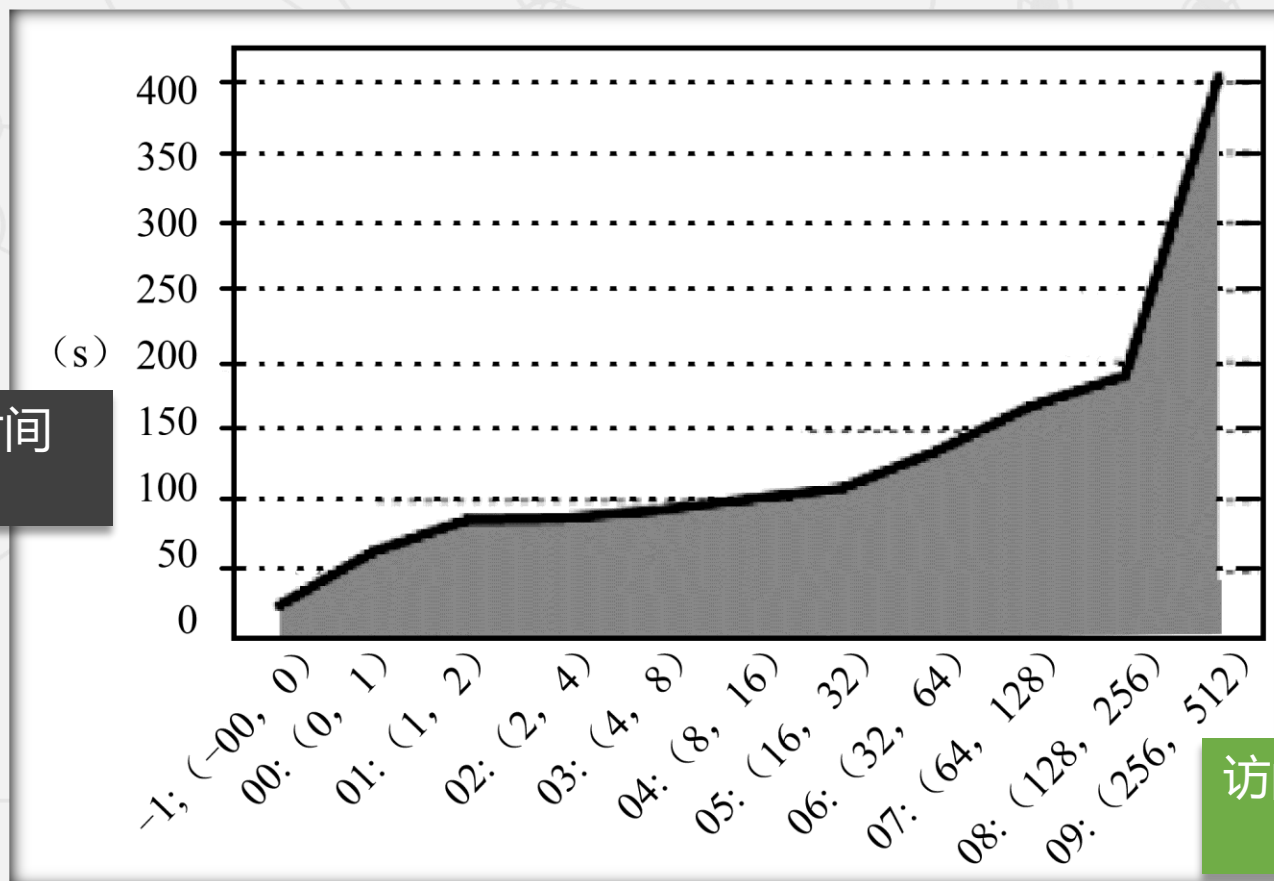
超过**70%**的查询是不需要从磁盘访问任何数据的

这些查询的平均访问延迟大约是**25秒**

96.5%的查询需要访问的磁盘量不超过**1GB**

2.8 内存大数据分析系统PowerDrill

● 性能分析与对比



2.8 内存大数据分析系统PowerDrill

• PowerDrill与Dremel的对比

	Dremel	PowerDrill
设计目标	处理非常大量的数据集	分析少量的核心数据集
设计理念	处理的数据来自外存	处理的数据尽可能地存于内存
	未进行数据分区，分析时要扫描所有需要的列	使用了组合范围分区，分析时可以跳过很多不需要的分区
	数据通常不需要加载，增加数据很方便	数据需要加载，增加数据相对不便